

MÓDULO VI

Introdução à Programação II

MÓDULO VI

Associatividade e precedência de operadores

Associatividade e precedência de operadores é um assunto um pouco marginalizado no estudo de linguagens de programação. Em algum momento todo desenvolvedor vai se deparar com um problema relacionado a isso.

Por exemplo, a expressão $8 * 2 - 1$ pode ser vista como $(8 * 2) - 1$ ou $8 * (2 - 1)$ e essa diferença muda a forma dela ser calculada e, conseqüentemente, o seu resultado. Essa ambigüidade foi resolvida lá nos primórdios pelos matemáticos através de regras de precedência e associatividade.

Essas mesmas regras são aplicadas nos projetos das linguagens de programação.

Quando um projetista de linguagem vai criar a especificação dela, ele precisa definir de antemão todas essas regras.

MÓDULO VI

Precedência: um operador possui maior precedência que outro se ele precisar ser analisado antes em todas as expressões sem parênteses que envolverem apenas os dois operadores. Na matemática, na expressão $8 * 2 - 1$, a multiplicação é sempre avaliada antes da subtração, ou seja, ela possui maior precedência. Portanto, a expressão equivalente seria: $(8 * 2) - 1$.

Já a **associatividade** especifica se os operadores de igual precedência devem ser avaliados da esquerda para a direita ou da direita para a esquerda. De volta à matemática, o operador “menos” possui associatividade à esquerda, portanto, $10 - 9 - 8$ é equivalente a $(10 - 9) - 8$.

MÓDULO VI

Na matemática

Precedência na matemática, da maior para a menor:

1. **Parênteses;**
2. **Expoentes;**
3. **Multiplicações e divisões;** *(da esquerda para a direita);*
4. **Somas e subtrações.** *(da esquerda para a direita);*

Grande parte das linguagens de programação seguem *(sempre que possível)* as convenções matemáticas comuns para associatividade e precedência.

Na maioria das linguagens de programação os operadores de atribuição são definidos com associatividade à direita. Ou seja, $\mathbf{a = b = c}$ é o equivalente a $\mathbf{a = (b = c)}$, que alimenta as variáveis $\mathbf{a}$ e $\mathbf{b}$ com o valor armazenado em $\mathbf{c}$. As linguagens de programação possuem em suas documentações uma tabela dos operadores e suas precedências, da maior para menor, bem como a associatividade destes operadores.

Portanto, a regra aqui é avaliar essa tabela na linguagem que você utiliza.

MÓDULO VI

Forçando precedência

Assim como na matemática, as linguagens permitem que usemos parênteses para forçar uma maior precedência. Essa expressão aqui:

```
x = 8 + 9 * 2; // 26
```

Em qualquer linguagem, vai resultar em 26, pois a multiplicação possui maior precedência que a subtração. No entanto, se utilizarmos parênteses entre a operação de soma, ela passa ter precedência maior que a multiplicação e, conseqüentemente, o resultado muda:

```
x = (8 + 9) * 2; // 34
```

MÓDULO VI

Sem uma definição clara de precedência as expressões com múltiplos operadores se tornariam ambíguas. Um exemplo disso:

```
x = 4/2*1+3; // 5
```

Que resultado você espera ter? Pra quem está lendo essa expressão, não é tão óbvio como calculá-la. Por isso forçar precedência é uma boa prática, traz mais legibilidade e coerência para sua operação. O resultado da expressão acima é diferente do resultado que força precedência:

```
x = (4/2)*(1+3); // 8
```

MÓDULO VI

Precedência é sobre agrupamento

Uma observação **muito importante** a ser feita é que precedência **não** determina a ordem de avaliação, precedência apenas determina como a operação vai ser agrupada para depois ser avaliada.

Ou seja, precedência especifica que a expressão:

```
f1() + f2() * f3()
```

Será agrupada como:

```
f1() + (f2() * f3())
```

MÓDULO VI

Ou seja, precedência não fala que a função **f2()** vai ser avaliada primeiro que **f1()**.

A ordem de avaliação padrão da maioria das linguagens de programação é da esquerda para a direita, mas as regras de associatividade podem alterar isso no meio do caminho.

Então, a ordem de avaliação das funções acima seria primeiro a **f1()**, depois a **f2()** e por fim a **f3()**, da esquerda para a direita. A função da precedência foi apenas agrupar as duas expressões **(f2() * f3())**. Abaixo exemplos de como diferentes expressões são agrupadas no processo de parsing de uma linguagem de programação (*na construção da árvore sintática abstrata*):

-a * b	=> ((-a) * b)
!-a	=> (!(-a))
a + b + c	=> ((a + b) + c)
a * b * c	=> ((a * b) * c)
a * b / c	=> ((a * b) / c)
a + b / c	=> (a + (b / c))
a + b * c + d / e - f	=> (((a + (b * c)) + (d / e)) - f)
3 + 4 * 5 == 3 * 1 + 4 * 5	=> ((3 + (4 * 5)) == ((3 * 1) + (4 * 5)))

MÓDULO VI

Por exemplo, a expressão:

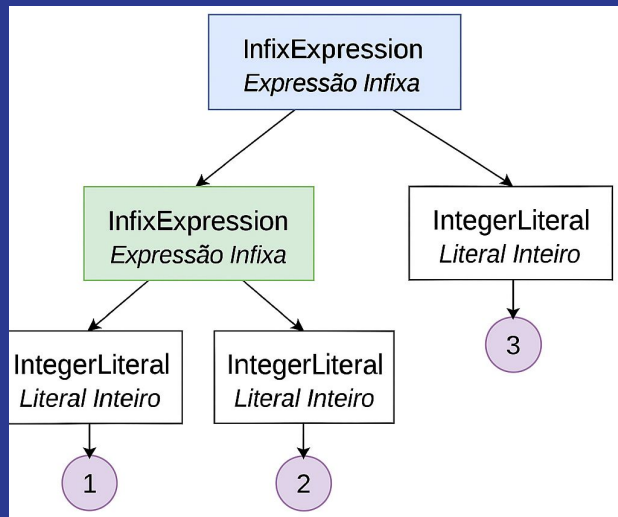
```
1 + 2 + 3;
```

No parsing da linguagem, depois de se aplicar as regras de precedência, ela é agrupada dessa forma:

```
((1 + 2) + 3)
```

MÓDULO VI

A representação da árvore abstrata dessa expressão seria algo como:



Ou seja, mais à esquerda temos uma expressão que um lado do seu nó é o literal 1 e o outro lado o 2.
E essa expressão está à esquerda da expressão que tem o nó do literal 3.

MÓDULO VI

Explicando a Árvore da Imagem:

1. Nó Raiz (*Topo da Árvore*)
 - **InfixExpression** (*Expressão Infixa*): Representa a operação matemática principal da expressão.
2. Filhos do Nó Raiz
 - **Esquerda**: Outra InfixExpression, indicando que há outra operação a ser resolvida primeiro.
 - **Direita**: Um IntegerLiteral (*Literal Inteiro*) contendo o número **3**.
3. Nós da Esquerda
 - Como temos uma expressão infixada, esta InfixExpression tem seus próprios filhos:
 - Dois IntegerLiteral (**1 e 2**) que representam os operandos dessa operação.
4. Nós Folha (*Finais*)
 - Os IntegerLiteral (**1, 2 e 3**) são os números da expressão. Eles são os nós finais da árvore.

MÓDULO VI

Tabela de precedência e associatividade

Na documentação da sua linguagem de programação, você deverá encontrar uma tabela parecida com essa abaixo, onde os operadores são ordenados pelos de maior precedência e uma coluna especificando a associatividade:

Nome	Operador(es)	Associatividade
Unário	!	direita
Aritmética	* / %	esquerda
Aritmética	- +	esquerda
Comparação	< > <= >=	esquerda
Comparação	== !=	esquerda
Atribuição	= += -= *=	direita

MÓDULO VI

Nessa especificação temos claro que a operação aritmética de **multiplicação** * tem precedência maior que a **subtração** - (*por estar numa ordem superior na tabela*). Da mesma forma, os operadores aritméticos possuem maior **precedência** que os operadores de **atribuição**.

O operador **unário** ! de negação tem associatividade à direita, ou seja, quer dizer que primeiro a expressão da direita será avaliada antes de fazer a negação. Da mesma forma os operadores de atribuição vão resolver as expressões à direita antes de fazer a atribuição.

Apesar de as linguagens de programação seguirem regras parecidas, não há como generalizar, há sempre algumas diferenças, portanto, não deixe de consultar a documentação da sua linguagem.

MÓDULO VI

Listas, Tuplas, Conjuntos e Dicionários

Listas:

- Coleção ordenada e mutável de elementos;
- Permite armazenar itens duplicados;
- Acessados através de índices (0, 1, 2, ...);
- Utiliza colchetes [] para criação;

Exemplo:

```
lista = ['palavra', True, 13, 3.2]  
print(lista) # ['palavra', True, 13, 3.2]
```

- Para acessar um item específico de sua lista, basta informar o índice correspondente dentro de colchetes:

```
lista = ('palavra', True, 13, 3.2)  
print(lista) # ['palavra', True, 13, 3.2]  
print(lista[1]) # True
```

MÓDULO VI

Tuplas:

- Coleção ordenada e imutável de elementos;
- Permite armazenar itens duplicados;
- Acessados através de índices (0, 1, 2, ...);
- Utiliza parênteses () para criação;

Exemplo:

```
tupla = ('palavra', True, 13, 3.2)
print(tupla) # ('palavra', True, 13, 3.2)
```

- Caso queira acessar um item específico de sua tupla, basta informar o índice correspondente dentro de colchetes:

```
tupla = ('palavra', True, 13, 3.2)
print(tupla) # ('palavra', True, 13, 3.2)
print(tupla[3]) # 3.2
```

MÓDULO VI

Dicionários:

- Coleção ordenada e mutável de pares chave-valor;
- Não permite chaves duplicadas, mas valores podem ser repetidos;
- Acessados através das chaves, não por índices;
- Utiliza chaves { } para criação;

Exemplo:

```
dicionario = {'string': 'palavra', 'logico': True, 'numero': 13, 'flutuante': 3.2}  
print(dicionario) # {'string': 'palavra', 'logico': True, 'numero': 13, 'flutuante': 3.2}
```

- Caso queira acessar um item específico de seu dicionário, basta informar a chave correspondente ao valor que deseja dentro de colchetes:

```
dicionario = {'string': 'palavra', 'logico': True, 'numero': 13, 'flutuante': 3.2}  
print(dicionario) # {'string': 'palavra', 'logico': True, 'numero': 13, 'flutuante': 3.2}  
print(dicionario['string']) # palavra
```


MÓDULO VI

Conjuntos (Sets):

- Coleção não ordenada e não indexada de elementos;
- Não permite itens duplicados;
- Não possui acesso através de índices ou chaves;
- Utiliza chaves { } para criação, mas sem pares chave-valor;

Exemplo:

```
conjunto = {'palavra', True, 13, 3.2}
print(conjunto) # {'palavra', True, 3.2, 13}
```

- Repare que a ordem de inserção dos itens foi mudada ao rodar o programa;
- Como não há acesso direto aos itens de um conjunto, ao informar um índice dentro de colchetes, como nos exemplos anteriores, o terminal retorna um erro:

```
conjunto = {'palavra', True, 13, 3.2}
print(conjunto)
print(conjunto[2])
```

MÓDULO VI

Conclusão

Cada tipo de coleção tem suas vantagens e usos específicos.

As listas são usadas quando a ordem dos elementos é importante e quando é necessário modificar a coleção após a criação.

As tuplas são úteis quando se quer uma coleção imutável, por exemplo, para definir coordenadas.

Os dicionários são ideais para associar valores a chaves e fazer buscas rápidas por essas chaves.

Por fim, os conjuntos são úteis quando não precisamos nos preocupar com a ordem e precisamos garantir que os itens sejam únicos.

De maneira simples, podemos dizer que as principais diferenças entre as coleções são:

- **Listas:** Ordenadas, mutáveis e permitem duplicados.
- **Tuplas:** Ordenadas, imutáveis e permitem duplicados.
- **Dicionários:** Ordenados, mutáveis e não permitem chaves duplicadas.
- **Conjuntos:** Não ordenados, não indexados e não permitem itens duplicados.

MÓDULO VI

Linguagem compilada e linguagem interpretada?

A **interpretação** ocorre quando o uso (comumente a execução) do código se dá junto à análise do mesmo.

A **compilação** é o processo de análise e possivelmente transformação do código fonte em código alvo, ou seja, o uso (execução, por exemplo) se dá em processo separado posterior, ainda que não tão posterior assim. Então aplicações que rodam **interpretadas precisam do código fonte enquanto que as compiladas só precisam do código alvo para funcionar.**

MÓDULO VI

Interpretação

A interpretação ocorre em cima do código fonte da aplicação escrita. Existem várias formas de proceder a interpretação, que é a análise do código no momento da execução. Pode ser feito em partes pequenas (linhas, por exemplo) ou em partes maiores, funções ou arquivos.

Neste processo a análise do código é feita todas as vezes que ele precisa executar, e os erros só são detectados durante a execução. Esta é uma diferença fundamental.

Interpretador

Obviamente que para executar código interpretado é necessário um software interpretador.

E, pasmem, ele pode ser interpretado também, ainda que não seja comum.

Esse software interpretador controla o fluxo do trabalho da interpretação e garante a execução do que for necessário.

MÓDULO VI

É comum a interpretação envolver um passo de compilação não só para a análise mas também para a transformação do código. Isto pode ocorrer por completo ou em partes. Mas ela ocorrerá sob demanda e ela ocorrerá por discricção do interpretador. Pra falar a verdade não é fácil fazer uma interpretação sem alguma forma de compilação, mesmo que rudimentar.

Em alguns casos esta compilação costuma ser chamada de *just in time*. Mas é importante diferenciar da compilação tradicional que tem uma função diferente. Eu prefiro considerar isso mais uma compilação que uma interpretação.

MÓDULO VI

Área nebulosa

É importante entender que a interpretação envolve um processo de compilação que pode até mesmo transformar um código fonte em outro código alvo. A grande diferença é que após esta compilação já existe uma ação ocorrendo. Então mesmo que ocorra uma transformação, erros só são detectados no momento da execução ou alguns instantes antes dela efetivamente se iniciar, mas dentro do mesmo processo.

Há quem ache que se existe este processo de compilação do fonte em código alvo, especialmente se for nativo à máquina que está suportando a execução, imediatamente anterior à execução faria isto ser considerado um código compilado. Sim, o código alvo é compilado, mas ele é transparente ao processo todo.

E esta compilação não pode se dedicar a otimizar o código sob pena de criar um custo maior que o ganho.

MÓDULO VI

Empacotamento

É possível encapsular código fonte em um arquivo único, possivelmente dentro de um executável, podendo compactá-lo e até mesmo criptografá-lo, mas ainda assim ter o processo de interpretação.

Vantagem

A maior e mais perceptível vantagem da interpretação é a facilidade e rapidez para iniciar a execução do código já escrito.

Algumas pessoas citam outras vantagens desta forma de execução de códigos, mas que na verdade a vantagem é colateral ou apenas relacionada, ou seja, é comum ter a vantagem em "linguagens interpretadas" mas na verdade é uma coincidência e a vantagem não ocorre pela interpretação em si. Outros casos citados são por desconhecimento que a forma compilada também pode ter as mesmas vantagens. Se uma linguagem normalmente compilada não fornece a vantagem, é um problema dela e não do método de execução.

MÓDULO VI

Uma destas supostas vantagens citadas costuma ser poder rodar em várias plataformas.

Códigos compilados também podem se não forem compilados para código nativo diretamente.

E códigos interpretados por interpretadores que só estão disponíveis para uma plataforma não tem esta flexibilidade. A vantagem não é uma propriedade da forma de execução.

Então pode-se falar em tendência, mas não uma garantia.

Do ponto de vista de implementação da linguagem é mais fácil fazer um interpretador, especialmente linguagens dinâmicas. Mas não acho que isto interesse para a maioria das pessoas, parte da facilidade é porque não se espera muito de linguagens que são pensadas mais para funcionarem de forma interpretada.

MÓDULO VI

Linguagem Interpretada → Receita Traduzida Passo a Passo

- Agora, imagine que você tem a mesma receita em inglês, mas, em vez de traduzir tudo antes, um **tradutor** (interpretador) lê cada linha e explica para você **na hora**, passo a passo.
- Se houver um erro em um passo, o processo para até aquele ponto.
- **Exemplo de linguagem:** Python, JavaScript.

MÓDULO VI

Compilação

A compilação tradicional considera partes maiores de código sempre, é preciso entender o todo do código. Não precisa ser feita em toda a aplicação mas em tudo o que é necessário em determinada parte do código.

A análise é feita de forma semelhante ao da interpretação, tanto que, muitas vezes o processo de interpretação no fundo é feito por um compilador. É claro que o funcionamento deste compilador é um pouco diferente no interpretador. Mas a análise léxica, sintática e semântica que todo compilador faz também é necessária no interpretador.

A grande diferença entre ambos é a forma como o resultado é gerado. A interpretação executa o código analisado. A compilação gera um outro código que será posteriormente usado (executado) por um ambiente que entenda o seu funcionamento. Pode ser uma máquina virtual ou uma máquina real.

MÓDULO VI

Execução

Então códigos compilados (em um código alvo) costumam rodar em cima de uma aplicação que simula uma máquina ou um sistema operacional, ou se o código for considerado nativo em cima de um sistema operacional real. Embora existam casos que códigos compilados gerem binários capazes de executar em uma máquina por conta própria, o mais comum é precisar de um software que controle a sua execução no geral (mas não controle as instruções), assim como códigos interpretados.

A diferença é a forma como a execução ocorre.

Códigos compilados para executáveis nativos podem ter seu código passado diretamente ao processador.

Executáveis com *bytecodes* intermediários podem ser compilados para código nativo ou executado através de uma decodificação e execução em laços (*loops*) da máquina virtual, o que é parecido com uma interpretação.

Compilações costumam gerar códigos de máquina nativos ou de ambientes virtuais, mas não necessariamente.

Eles podem gerar outros códigos fonte ou outras formas que eles podem ser usados posteriormente.

Há quem diga que só há código compilado quando ele é nativo.

Existem casos em que o processo de compilação gera um código intermediário que depois poderá ser compilado novamente. A segunda compilação normalmente é chamada de *just-in-time compilation* feita por um JITter. Mas esta compilação é diferente. A compilação não precisa necessariamente gerar nada, ela pode ser feita apenas como verificação ou pode gerar um código que não será executado.

MÓDULO VI

Empacotamento

Após a compilação existem casos onde se faz a *linkedição* que gera um arquivo executável com tudo o que precisa para executar e em um formato suportado por algum sistema operacional. Mas isso não é verdade em vários casos que utilizam máquinas virtuais.

MÓDULO VI

Vantagens

Uma vantagem clara do uso de códigos compilados é que uma parte considerável dos erros de programação podem ser evitados no processo de compilação e nunca chegam para a execução.

Códigos compilados são mais fáceis de proteger a propriedade intelectual, especialmente quando se distribui apenas o código nativo. Mas é possível fazer o mesmo com códigos a serem interpretados, depende de implementação. Proteção completa não existe em nenhum caso, apenas a facilidade maior é uma vantagem.

Outra vantagem é que códigos compilados costumam executar mais rapidamente, já que a análise não precisa ser feita durante a execução, além do que, por analisar o código de forma integral, pode fazer otimizações. Algumas destas otimizações demoram para serem realizadas pelo compilador, mas como este trabalho de otimização não atrasará a execução ele pode ser feito confortável e agressivamente.

MÓDULO VI

Velocidade

Costuma haver uma confusão com o ganho de velocidade de algumas linguagens. Embora as duas melhorias citadas acima sejam importantes para dar velocidade para aplicações, muitas vezes um fator muito importante é que "linguagens compiladas" tendem ter características próprias que ajudam a velocidade por si só.

Por exemplo elas costumam usar tipagem estática e não dinâmica, como é comum nas linguagens interpretadas. Claro que o oposto pode ocorrer, mas como é comum haver uma relação entre esses tipos de linguagens, há a confusão entre os mais leigos no assunto.

Outro ponto importante é que linguagens que oferecem acesso mais baixo nível dão melhor atenção a certos detalhes e melhor controle sobre o código, e portanto oferecem melhor performance, costumam ser compiladas, mas isto é uma coincidência não uma característica inerente.

MÓDULO VI

Algumas pessoas vão dizer que a compilação gera código nativo e por ele rodar diretamente há ganho de performance. Isto é verdade, mas é um efeito colateral. *Assembly* não é compilado (*é montado, como o próprio nome diz, tá, é verdade que alguns até são compilados*) e gera código nativo que é rápido. Não é o processo da compilação que faz o código executar mais rápido, neste caso.

Isto costuma ser verdade, no entanto é possível executar códigos interpretados mais rapidamente por ele oferecer melhor contexto, apesar que não é fácil, não acontece sempre.

Então não é possível executar um código em Python mais rápido que um mesmo código escrito em C++, se ambos forem bem escritos e não for algo feito especificamente para mostrar um *corner case* que não tem relevância no mundo real (eu sei que alguém vai querer mostrar que tem algum jeito maluco, muito específico que Python consegue ser mais rápido). Mesmo que o código Python seja compilado (e isto é possível).

MÓDULO VI

JavaScript

JavaScript é interpretada ou compilada? Nem estou falando do Rhino ou JScript que são obviamente compilados e depois executados em cima da JVM e CLR (library), respectivamente.

Falo do código que roda nos navegadores.

Hoje todos os bons navegadores compilam o código fonte em código nativo, de uma forma ou de outra, mas precisa fazer a análise do código fonte, que obviamente é necessário, todas as vezes antes de executar, e os erros serão verificados logo no momento imediato ao da execução. Isto é interpretação.

A transformação em código nativo é só uma otimização.

MÓDULO VI

Em geral os *engines* dos navegadores costumam executar códigos mais rapidamente que estas implementações que compilam o código JS.

Alguém pode dizer: e se os navegadores fizerem cache do código nativo? Talvez já o façam, não sei como os *engines* modernos estão operando. Eu acho que o cache aí é apenas uma estratégia de otimização do interpretador. O código nativo pode desaparecer, ficar invalidado, etc. Ele não é o foco e é opcional.

Mas a execução é compilada, então usar este termo não é exatamente errado. Mas o código da sua aplicação, estritamente falando, passa por um processo de interpretação.

MÓDULO VI

Linguagem Compilada → Receita de Bolo Traduzida

- Imagine que você tem uma receita de bolo escrita em inglês, mas só entende português.
- Antes de começar a cozinhar, você **traduz toda a receita** para o português de uma só vez (isso é a **compilação**).
- Depois, basta seguir os passos já traduzidos (o programa compilado) sem precisar interpretar de novo.
- **Exemplo de linguagem:** C, C++.

MÓDULO VI

Conclusão

- **Compilada:** Traduz tudo de uma vez antes de executar (rápido na execução, mas precisa ser recompilado para mudanças).
- **Interpretada:** Traduz e executa linha por linha (mais lento, mas permite testes rápidos sem recompilar).

Estão aí definições, comparações, etc. Mas nem conseguimos definir claramente quando um código está sendo interpretado ou compilado. Ou seja, isto tem importância limitada. Especialmente porque a interpretação é enormemente beneficiada pela compilação e códigos compilados podem eventualmente se comportar como códigos interpretados, aplicando-se as devidas técnicas.

Se levar ao pé da letra algumas definições encontradas por aí, praticamente não existem mais interpretadores.

MÓDULO VI

Modularização

O que são Parâmetros, Argumentos e Retornos em Funções no Javascript?

- **Parâmetro** de uma **função**, é uma ou mais informações que a função precisa para funcionar corretamente.
- **Retorno** de uma **função** é a informação que a sua **função** vai retornar após ter feito todo o procedimento.
- **Argumento** de uma **função**, nada mais é do que as informações que você vai passar para a execução da sua **função**, que estão relacionadas com os **parâmetros**.

Então se para rodar a **função** eu preciso de **2 parâmetros**, na hora de chamar a **função** eu preciso passar **2 argumentos**, um para cada **parâmetro** para que ela funcione.

Para relembrar, podemos dizer que função é uma sequência de passos para executar determinada ação, a essa sequência de passo damos um nome que será usado posteriormente sempre que quisermos que a ação seja refeita.

MÓDULO VI



A Educação é o primeiro passo para um futuro melhor...

MÓDULO VI

Parâmetros

Parâmetros



```
function fazerPizza(tipoDoQueijo, tipoDoRecheio) {  
  const pizza = 'Pizza de ${tipoDoQueijo} com ${tipoDoRecheio}';  
}
```

Observe que dentro dos parênteses vão estar os parâmetros da função, neste caso esses parâmetros representam os sabores da pizza que está sendo solicitada, colocar esse valor em um parâmetro é melhor do que colocar os sabores diretamente nos parênteses, assim alterando os parâmetros conseguimos alterar os sabores sem ter que mexer na estrutura da função.

MÓDULO VI

Retorno

```
function fazerPizza(tipoDoQueijo, tipoDoRecheio) {  
  const pizza = 'Pizza de ${tipoDoQueijo} com ${tipoDoRecheio}';  
  return pizza;  
}
```



Retorno

Toda vez que a sequência de passos da função geram um resultado chamamos isso de retorno, representado na função pelo return, esse elemento é necessário na estrutura da função para sabermos o resultado que está sendo gerado.

MÓDULO VI

Argumentos

Argumentos



```
const pizzaDoCliente = fazerPizza ('Brie', 'Damasco');  
console.log(pizzaDoCliente);
```

Por fim, temos os **argumentos**, aqui já estamos chamando a **função** pelo nome “*fazerPizza*” para que o passo a passo seja executado, porém, temos que colocar qual valor ou, neste caso, “*sabor de pizza*” a **função** vai rodar agora, esse valor passado nos parênteses chamamos de **argumentos**.

Então chamamos a **função** -> colocamos os **argumentos** e a **função** executa os passos com esses valores que acabamos de informar, esses valores podem ser trocados todas às vezes que precisarmos executar essa **função** e a **função** entende que precisa a cada vez que for chamada substituir pelos **argumentos** novos.

MÓDULO VI

O que é recursão?

Em desenvolvimento de software, **recursão** é quando você tem uma chamada para um método ou função dela para ela mesma.

Você já colocou um espelho em frente ao outro? A imagem de um refletindo no outro, que reflete no um, que reflete no outro, que reflete... então, isso é um exemplo de **recursão** :)

Suponha que você tenha uma função chamada **recurse()**. Ela será uma função **recursiva** caso ela mesma se invoque dentro do seu escopo, dessa forma:

```
function recurse() {  
    // ...  
    recurse();  
    // ...  
}
```

MÓDULO VI

Uma função recursiva sempre precisa ter uma condição que vai parar de invocar ela mesma, caso contrário ela se chamará infinitamente.

Portanto, uma função recursiva normalmente se parece com algo assim:

```
function recurse() {  
    if(condition) {  
        // stop calling itself  
        //...  
    } else {  
        recurse();  
    }  
}
```

Geralmente, as funções recursivas são usadas para dividir um grande problema em problemas menores. Você pode descobrir que elas são muito usadas nas estruturas de dados, como árvores binárias, gráficos e algoritmos de pesquisas binárias e classificação rápida.

MÓDULO VI

Exemplos de funções recursivas no JavaScript

Vamos dar alguns exemplos de como usar as funções recursivas.

1) Um exemplo simples de função recursiva

Suponha que você precise desenvolver uma função que faça a contagem regressiva de um número determinado até chegar em 1. Por exemplo, para fazer a contagem regressiva de 3 até 1:

```
3  
2  
1
```

MÓDULO VI

Confira a função `countDown()` a seguir:

```
function countDown(fromNumber) {  
    console.log(fromNumber);  
}  
  
countDown(3);
```

No momento a função `countDown()` exibe apenas o número 3.
Para fazer a contagem regressiva do número 3 ao 1, você pode fazer assim:

- Mostre o número 3;
- e chame `countDown(2)` que vai mostrar o número 2;
- e chame `countDown(1)` que vai mostrar o número 1.

MÓDULO VI

O código a seguir altera a função `countDown()` para uma função recursiva:

```
function countDown(fromNumber) {  
  console.log(fromNumber);  
  countDown(fromNumber-1);  
}  
  
countDown(3);
```

A chamada `countDown(3)` será executada até que o tamanho máximo da pilha de chamadas seja excedido, mostrando um erro assim:

```
Uncaught RangeError: Maximum call stack size exceeded.
```

MÓDULO VI

Isso porque a função não tem uma condição para parar de se chamar, tornando a execução em um loop infinito.

A contagem regressiva deveria parar até quando o próximo número fosse zero, para isso, adicionamos uma condição if dessa forma:

```
function countdown(fromNumber) {  
  console.log(fromNumber);  
  
  let nextNumber = fromNumber - 1;  
  
  if (nextNumber > 0) {  
    countdown(nextNumber);  
  }  
}  
  
countdown(3);
```

MÓDULO VI

Resultado:

```
3  
2  
1
```

Agora a função **countDown()** funciona como o esperado.

No entanto, conforme mencionado no tutorial de Tipo Função, o nome da função é uma referência ao objeto de função real.

Se em algum lugar do código, o nome da função for definido como null, a função recursiva irá parar de funcionar.

MÓDULO VI

Por exemplo, o código a seguir resultará em um erro:

```
let newYearCountDown = countdown;  
  
// somewhere in the code  
  
countDown = null;  
  
// the following function call will cause an error  
newYearCountDown(10);
```

Erro:

```
Uncaught TypeError: countDown is not a function
```


MÓDULO VI

Como esse script funciona:

- *Primeiro*, atribua o nome da **função countdown** à variável **newYearCountDown**.
- *Segundo*, defina a referência da **função countdown** como **null**.
- *Terceiro*, chame a função **newYearCountDown**.

O código causa um erro porque o corpo da **função countdown()** faz referência ao nome da **função countdown** que foi definido como **null** no momento da chamada da **função**.

MÓDULO VI

Para corrigir isso, você pode usar uma expressão de função nomeada da seguinte forma:

```
let countdown = function f(fromNumber) {  
  console.log(fromNumber);  
  
  let nextNumber = fromNumber - 1;  
  
  if (nextNumber > 0) {  
    f(nextNumber);  
  }  
}  
  
let newYearCountDown = countdown;  
countdown = null;  
newYearCountDown(10);
```

MÓDULO VI

2) Calcular a soma dos dígitos de um número

Dado um número qualquer, por exemplo 324, calcule a soma dos dígitos:

$$3 + 2 + 4 = 9$$

Para aplicar a técnica recursiva, você pode usar as seguintes etapas:

```
f(324) = 4 + f(32)
f(32)  = 2 + f(3)
f(3)   = 3 + 0 (stop here)
```

MÓDULO VI

Então:

```
f(324) = 4 + f(32)
f(324) = 4 + 2 + f(3)
f(324) = 4 + 2 + 3
```

O seguinte código ilustra a função recursiva **sumOfDigits()**:

```
function sumOfDigits(num) {
  if (num == 0) {
    return 0;
  }
  return num % 10 + sumOfDigits(Math.floor(num / 10));
}
```

MÓDULO VI

Como isso funciona:

- O **num%10** retorna o resto da divisão de num por **10**, por exemplo:
 $324 / 10 = 32$ e resta **4** portanto $324 \% 10 = 4$
- O **Math.floor(num/10)** retorna a parte inteira de num dividido por **10**:
 $324 / 10 = 32,40$ portando **32** é a parte inteira.
- O **if(num==0)** é a condição que interrompe a chamada da função.

Resumo

- Uma função recursiva é uma função que chama a si mesma até que seja interrompida .
- Uma função recursiva sempre tem uma condição que impede a função de chamar a si mesma em algum momento.

MÓDULO VI

Exemplos:

📌 1. Contagem Regressiva (Como um Foguete)

Imagine uma contagem regressiva para lançar um foguete. Em vez de usar **for** ou **while**, a função **chama a si mesma** até chegar a zero:

```
function foguete(contagem) {  
  if (contagem === 0) { // Caso base (condição de parada)  
    console.log("Decolar! 🚀");  
  } else {  
    console.log(contagem + "...");  
    foguete(contagem - 1); // Chamada recursiva  
  }  
}  
  
foguete(5); // Teste: 5... 4... 3... 2... 1... Decolar! 🚀
```

Explicação:

- A função se repete, diminuindo contagem até chegar a 0 (caso base).
- Se não tivesse o if, entraria em **loop infinito!**

MÓDULO VI

📌 2. Cálculo de Fatorial (Como uma Escada de Multiplicação)

Fatorial (ex: $5! = 5 \times 4 \times 3 \times 2 \times 1$) é um clássico exemplo recursivo:

```
function fatorial(n) {  
  if (n === 1) { // Caso base  
    return 1;  
  } else {  
    return n * fatorial(n - 1); // Chamada recursiva  
  }  
}  
  
console.log(fatorial(5)); // 120 (5! = 5 × 4 × 3 × 2 × 1)
```

MÓDULO VI

Explicação:

- A função "desce" até $n = 1$ e depois "sobe" multiplicando os resultados.
- Visualização:

```
fatorial(5) → 5 * fatorial(4)
           → 4 * fatorial(3)
           → 3 * fatorial(2)
           → 2 * fatorial(1)
           → 1 (para aqui!)
```



Dica Importante:

Recursão precisa **sempre** de:

1. **Caso base** (condição de parada → senão, vira loop infinito!).
2. **Chamada recursiva** (que deve avançar em direção ao caso base).

MÓDULO VI

Invertendo uma string

```
function inverter(str) {  
  if (str.length === 0) return ''  
  return str[str.length - 1] + inverter(str.substr(0, str.length - 1))  
}  
  
inverter('abcdefg');  
  
'gfedcba'
```

Explicação:

1. Definição da Função:

JavaScript

```
function inverter(str) {  
  // ... código da função ...  
}
```

- **function inverter(str):** Esta linha declara uma função chamada inverter que recebe um único argumento: uma string chamada **str**.

MÓDULO VI

2. Caso Base (Condição de Parada):

JavaScript

```
if (str.length === 0) return ''
```

- **if (str.length === 0):** Esta é a condição de parada da recursão. Ela verifica se o comprimento da string **str** é igual a zero (ou seja, se a string está vazia).
- **return '':** Se a string estiver vazia, a função retorna uma string vazia. Este é o ponto onde a cadeia de chamadas recursivas começa a se desfazer.

MÓDULO VI

3. Passo Recursivo:

JavaScript

```
return str[str.length - 1] + inverter(str.substr(0, str.length - 1))
```

- **str[str.length - 1]**: Isso acessa o último caractere da string **str**. Em JavaScript, os índices de string começam em **0**, então **str.length - 1** sempre será o índice do último caractere.
- **str.substr(0, str.length - 1)**: Este método extrai uma substring de **str** que começa no índice **0** e vai até o índice **str.length - 2**. Em outras palavras, ele cria uma nova string contendo todos os caracteres de **str** *exceto* o último.
- **inverter(...)**: Aqui está a chamada recursiva. A função **inverter** é chamada novamente, mas desta vez com a substring menor (*sem o último caractere*).
- **... + ...**: O último caractere da string original é concatenado (*adicionado*) ao resultado da chamada recursiva da função com a substring menor.

MÓDULO VI

Em resumo:

A função `inverter` funciona da seguinte maneira:

1. Ela pega o último caractere da **string**.
2. Ela chama a si mesma com o restante da **string** (*sem o último caractere*).
3. Ela concatena o último caractere com o resultado da chamada recursiva.
4. O processo continua até que a **string** se torne vazia (*caso base*), momento em que uma **string** vazia é retornada e as chamadas anteriores começam a construir a **string** invertida.

Embora a recursão seja uma forma elegante de resolver certos problemas, em alguns casos (*como a inversão de strings em JavaScript*), uma solução iterativa (*usando um loop*) pode ser mais eficiente em termos de desempenho e uso de memória. No entanto, entender a recursão é fundamental na ciência da computação.

MÓDULO VI

EXERCÍCIOS...

MÓDULO VI

FIM...